



1. What is a Function?

A function is a reusable block of code that performs a task.



2. Why Functions Matter

- They organize code.
- They reduce repetition.
- They make programs easier to read.
- They can accept input and return output.

3. function

fn

Starts a function definition.

```
function greet() {  
  ...  
}
```

4. call

call

Runs the function.

```
greet();
```

5. parameter

param

Input name inside the function.

```
function greet(name) {  
  ...  
}
```

6. argument

arg

Actual value passed in.

```
greet("Ayla");
```

7. return

ret

Sends a value back.

```
return total;
```





8. Function Syntax

Use function to start a function block.

```
function greet() {  
  console.log("Hello");  
}
```



9. Calling a Function

Run the function by its name.

```
greet();
```



10. Parameters

Receive input values.

```
function greet(name) {  
  ...  
}
```



11. Arguments

Send values into a function.

```
greet("Ayla");
```



12. Return Value

Sends the result back.

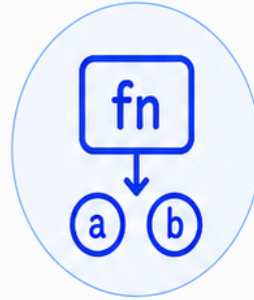
```
return price * quantity;
```



13. Reusable Code

Write once,
use many times.

```
greet("Ayla");  
greet("Ali");
```



14. Function with Parameters

Use inputs
inside the function.

```
add(a, b)
```



15. Function with Return

Returns a result
to the caller.

```
let total = add(5, 3);
```



16. Combining Inputs

Functions can work
with multiple values.

```
price * quantity
```



1. Function Declaration

Standard function form.

```
function sayHi() {  
  console.log("Hi");  
}
```



2. Function Expression

Store a function in a variable.

```
const sayHi = function() {  
  ...  
};
```



3. Arrow Function

A shorter modern syntax.

```
const add = (a, b) => a + b;
```



4. Function Names

Use clear verb-based names.

```
calculateTotal()  
checkLogin()
```



Good names improve clarity.





1. String Output

Functions can return text.

```
function greet(name) {  
  return "Hello " + name;  
}
```



2. Changing a Value

Functions can compute a new value.

```
function addBonus(score) {  
  return score + 5;  
}
```



New value: 15



3. Quick Example

Check a shopping total.

```
function total(price, quantity) {  
  return price * quantity;  
}
```

total = 30



4. Common Mistakes

- Forgetting to call the function
- Missing return
- Wrong parameter count
- Using unclear names



Small function mistakes can change your result.



1. Practice

Try these steps yourself:

- Create a function `add(a, b)`.
- Create a function `welcome(name)`.
- Use `return` in one function.
- Print the results with `console.log()`.

```
let a = 8;  
let b = 2;
```



2. Challenge Answer

If the code is:

```
function checkScore(score) {  
  if (score >= 60) return "Passed";  
  else return "Failed";  
}  
let result = checkScore(82);
```

- `result` -> `"Passed"`



3. Mini Project

Build a simple Shopping Total calculator.

```
const productName = "Book";  
function calculateTotal(price, quantity) {  
  return price * quantity;  
}  
let total = calculateTotal(20, 3);  
console.log(productName + " total: " + total);
```



4. Key Takeaway

- Functions group reusable code.
- Parameters receive input.
- Arguments pass values.
- `Return` sends results back.
- Clear names help readability.